SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

Authentication System 02

Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

[지난 시간] Login

Logout

회원 가입

회원 탈퇴

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

인증된 사용자에 대한 접근 제한

- is_authenticated 속성
- login_required 데코레이터

참고

- is_authenticated 코드
- 회원가입 후 자동 로그인
- 회원 탈퇴 개선
- Auth built-in form 코드

학습 시작

회원제 도서 웹사이트를 만들려고 합니다. 단순히 '로그인' 기능 하나만 있으면 될까요?사용자가 우리 서비스에 들어오고 나가는 전체 흐름을 생각해봅시다.

1. 비회원 (Anonymous User)
   - 아직 회원이 아니라면 회원가입 페이지를 통해 새로운 계정을 생성할 수 있어야 합니다.
   - 계정이 있다면 로그인 페이지를 통해 자신을 인증하고 접속할 수 있어야 합니다.
2. 회원 (Logged-in User)
   - 서비스 이용을 마치면 언제든 로그아웃하여 연결을 끊을 수 있어야 합니다.
   - 더 이상 서비스를 이용하지 않을 경우, 회원 탈퇴를 통해 자신의 정보를 삭제할 수 있어야 합니다.

이번 학습을 통해 우리는 사용자가 '누구인지'를 확인하고, 데이터의 생성(가입)부터 소멸(탈퇴)까지

사용자의 전체 생명주기를 관리하는 핵심적인 인증(Authentication) 시스템을 구축하게 될 것입니다.

학습 목표

- ✓ logout 함수를 사용하여 로그아웃 기능을 구현할 수 있다.
- ✓ 커스텀 User 모델을 위한 UserCreationForm을 작성한다.
- ✓ `get_user_model` 함수로 활성화된 User 모델을 참조한다.
- ✓ 커스텀 회원가입 폼을 활용하여 회원가입 기능을 구현한다.
- ✓ `request.user.delete`를 호출하여 회원 탈퇴 기능을 구현한다.
- ✓ `is_authenticated` 속성으로 인증 여부에 따라 접근을 제어한다.
- ✓ `login_required` 데코레이터로 인증된 사용자만 접근을 허용한다.

[지난 시간] Login

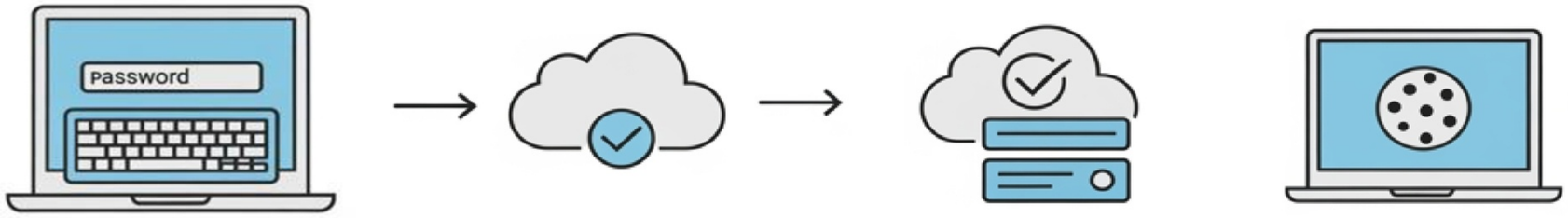
Login

- 클라이언트와 서버 간의 상태 정보를 유지하기 위해서 쿠키와 세션을 사용
- 서버는 수많은 요청이 각각 누구에게서 왔는지 구별해야 함
- 서버에 "나"임을 인증하는 과정이 바로 "Login"

Session
서버 측에서 생성되어 클라이언트와 서버 간의 상태를 유지, 상태 정보를 저장하는 데이터 저장 방식

<개념1_Django_Authentication System 01_세션>

➢결국 Login은 인증(id/password)을 완료한 뒤, 서버에 Session을 생성(Create)하고, 그 Session을 찾을 수 있는 Session ID를 쿠키에 담아 클라이언트에게 전달하는 것



1. 로그인(id/password)
2. 로그인 인증
3. 세션 생성 후 서버 저장
4. 서버가 세션을 만들어 클라이언트와 연결하는 과정

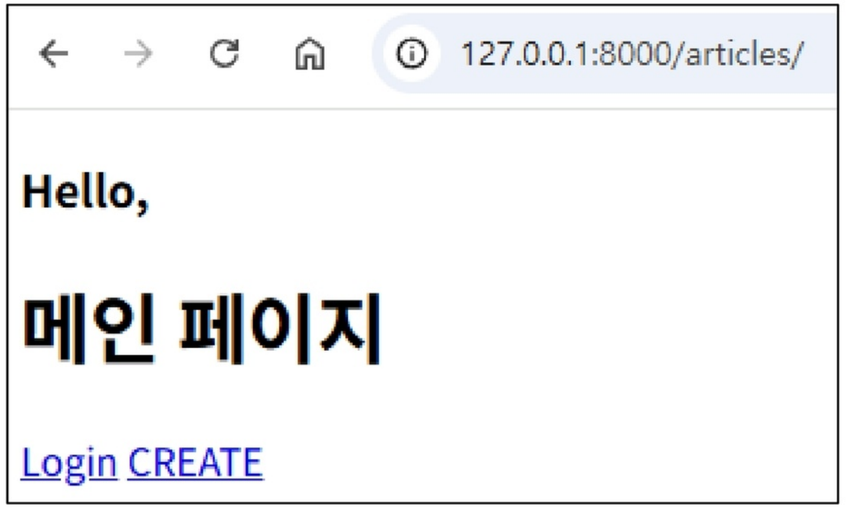
<그림13_Django_Authentication System 01_로그인 과정>

로그인 기능 구현 (1/4)

- 로그인 경로 url 및 링크 생성

```python
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    path('login/', views.login, name='login'),
]
```

```html
<!-- articles/index.html -->

<h1>메인 페이지</h1>
<a href="{% url "accounts:login" %}">Login</a>
```



<그림1_Django_Authentication System 02_로그인 화면>

로그인 기능 구현 (2/4)

- .../accounts/login/ url로 요청이 들어왔을 때 실행할 login 함수 작성
- 로그인 인증에 사용할 데이터를 입력 받는 built-in form (AuthenticationForm) 사용

```python
# accounts/views.py

from django.contrib.auth.forms import AuthenticationForm

def login(request):
    if request.method == 'POST':
        pass
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

TIP

로그인 view가 해야 하는 일은 2가지
1. 로그인 입력 페이지 제공 => 페이지를 조회하는 요청이므로 GET
2. 사용자 정보를 받아 실제 로그인(세션 생성) => 세션을 만드는 요청이므로 POST

➢ 게시글 생성(create) view와 똑같습니다. 결국 로그인은 세션을 생성(Create)하는 기능입니다.

AuthenticationForm()

- 로그인 인증에 사용할 데이터를 입력 받는 built-in form
- User 모델과 직접 연결된 “ModelForm”이 아닌, 일반 “Form”을 상속 받음
  - 일반 “Form”이기 때문에 사용자를 생성하거나 수정하는 용도가 아닌 “인증”하는 역할만 수행
  - `ArticleForm`과 부모 클래스가 다르기 때문에, 생성자 함수의 인자 구성도 다를 수 있다는 점을 예상해야 함

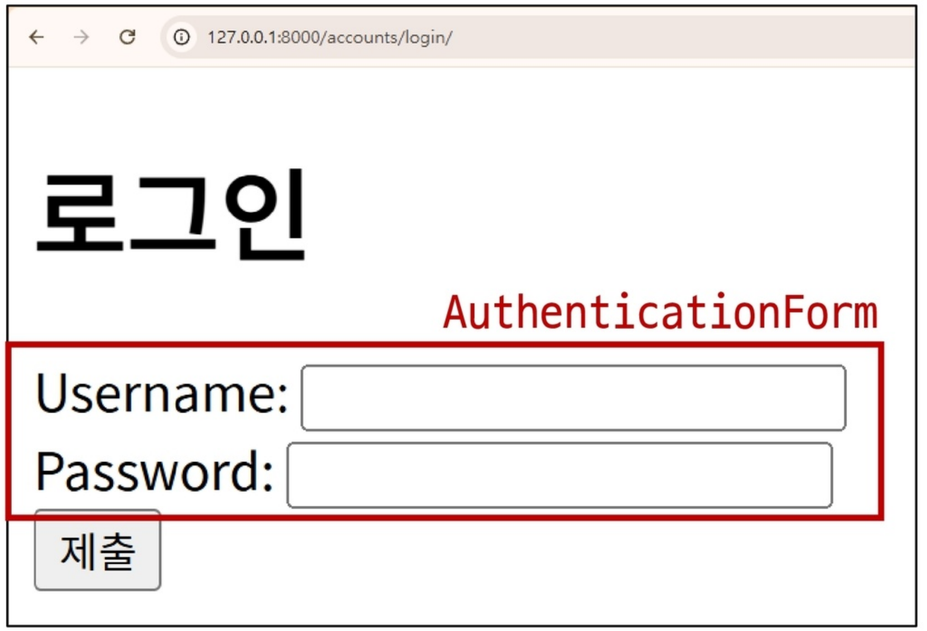
```
class AuthenticationForm(forms.Form):
    """
    Base class for authenticating users. Extend this to get a form that accepts
    username/password logins.
    """

    username = UsernameField(widget=forms.TextInput(attrs={"autofocus": True}))
    password = forms.CharField(
        label=_("Password"),
        strip=False,
        widget=forms.PasswordInput(attrs={"autocomplete": "current-password"}),
    )
```

<그림14_Django_Authentication System 01_AuthenticationForm>

로그인 기능 구현 (3/4)

- 로그인은 세션을 생성하는(서버 상태를 바꾸는) 요청이므로 POST 방식을 사용 (비밀번호가 URL이나 브라우저 기록에 남지 않는 효과도 있음)
- CSRF 공격을 방지하기 위해 `csrf_token` 작성
- 서버로부터 전달받은 `AuthenticationForm`을 화면에 출력

```
<!-- accounts/login.html -->

<h1>로그인</h1>
<form action="{% url 'accounts:login' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```



<그림15_Django_Authentication System 01_로그인 화면>

로그인 기능 구현 (4/4)

- 사용자가 입력한 로그인 정보를 입력 받는 로직을 “POST” 조건문에 작성
- 입력 받은 정보를 기반으로 로그인하여 세션을 만드는 “login” 함수를 활용

```python
# accounts/views.py
from django.shortcuts import render, redirect
from django.contrib.auth import login as auth_login

def login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, request.POST)
        # form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            auth_login(request, user)
            return redirect('articles:index')
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

```python
if form.is_valid():
    # 유효성 검사를 통과하면 인증된 유저 객체를 반환
    user = form.get_user()
    # 세션 생성
    auth_login(request, user)
    return redirect('articles:index')
```

AuthenticationForm의 첫 번째 인자가 request인 이유

```
form = AuthenticationForm(request, request.POST)
```

- `ArticleForm(request.POST)` ➔ `ModelForm`은 첫 번째 인자가 `data`
- `AuthenticationForm(request, request.POST)` ➔ `AuthenticationForm`은 첫 번째 인자가 `request`, 두 번째가 `data`

➢ `AuthenticationForm(request.POST)`로 쓰면 에러 없이 로그인만 안 됩니다.
`request.POST`가 `request` 자리에 들어가고 `data`는 비어 있게 되어, `is_valid()`가 조용히 `False`가 되기 때문입니다.

login(request, user)

- AuthenticationForm을 통해 인증된 사용자를 로그인 하는 함수
- request
  - 현재 사용자의 세션 정보에 접근하기 위해 사용
- user
  - 어떤 사용자가 로그인 되었는지를 기록하기 위해 사용

get_user()

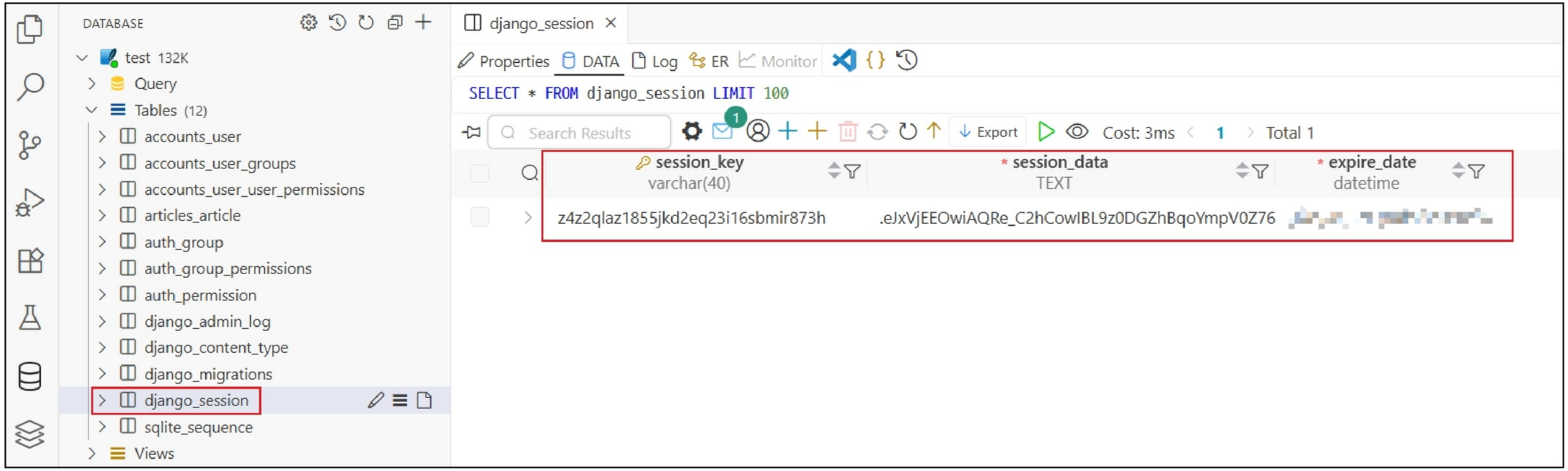
- AuthenticationForm의 인스턴스 메서드

```python
# accounts/views.py

from django.shortcuts import render, redirect
from django.contrib.auth import login as auth_login

def login(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, request.POST)
        # form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            auth_login(request, user)
            return redirect('articles:index')
    else:
        form = AuthenticationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/login.html', context)
```

➢ 유효성 검사를 통과했을 경우, 로그인 한 사용자 객체를 반환

세션 데이터 확인하기 (1/2)

1. 로그인 후 발급받은 세션 확인
   - `django_session` 테이블에서 확인

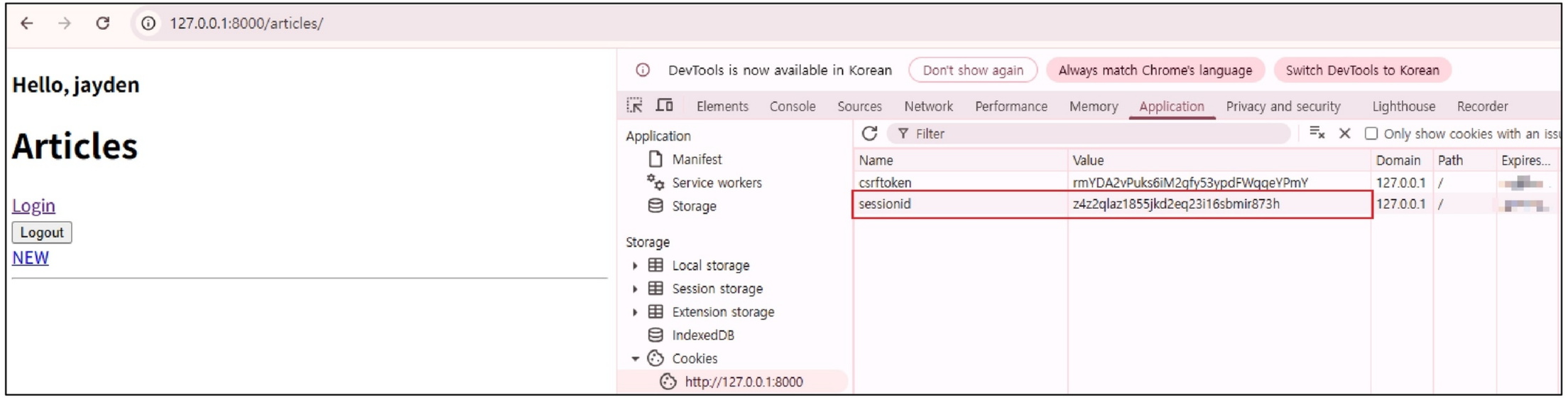


<그림4_Django_Authentication System 02_세션 DB 확인>

세션 데이터 확인하기 (2/2)

2. 브라우저에서 확인

- 개발자도구 - Application - Cookies



<그림5_Django_Authentication System 02_세션 개발자 도구 확인>

Logout

Logout

- 로그아웃은 Session을 Delete하는 과정
- 서버의 세션 데이터를 비우고, 클라이언트의 세션 쿠키를 삭제

Cookie
서버가 사용자의 웹 브라우저에
전송하는 작은 데이터 조각

<개념2_Django_Authentication System 02_쿠키>

logout(request)

1. DB에서 현재 요청에 대한 Session Data를 삭제
2. 클라이언트의 쿠키에서도 Session Id를 삭제

로그아웃 기능 구현 (1/3)

- 로그아웃 경로 url 및 링크 생성

```
# accounts/urls.py

urlpatterns = [
    path('login/', views.login, name='login'),
    path('logout/', views.logout, name='logout'),
]
```

로그아웃 기능 구현 (2/3)

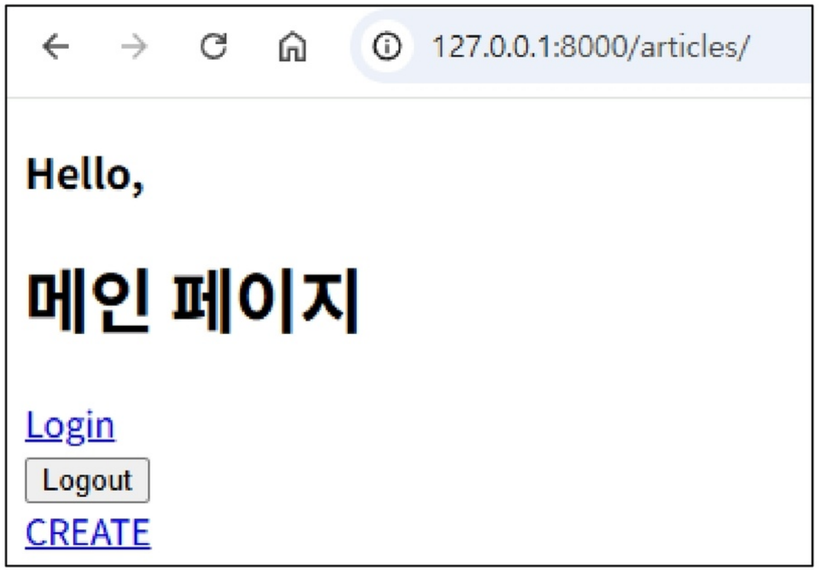
- .../accounts/logout/ url로 요청이 들어왔을 때 실행할 logout 함수 작성
- DB에서 현재 요청에 대한 Session Data를 삭제하고, 클라이언트의 쿠키에서도 Session Id를 삭제하는 내장 logout 함수 작성

```
from django.contrib.auth import logout as auth_logout

def logout(request):
    auth_logout(request)
    return redirect('articles:index')
```

로그아웃 기능 구현 (3/3)

- 로그아웃은 세션을 삭제하는(서버 상태를 바꾸는) 요청이므로 POST 방식을 사용
- CSRF 공격을 방지하기 위해 csrf_token 작성

```
<!-- articles/index.html -->

<h1>메인 페이지</h1>
<a href="{% url "accounts:login" %}">Login</a>
<form action="{% url "accounts:logout" %}" method="POST">
  {% csrf_token %}
  <input type="submit" value="Logout">
</form>
```



<그림6_Django_Authentication System 02_로그아웃 링크>

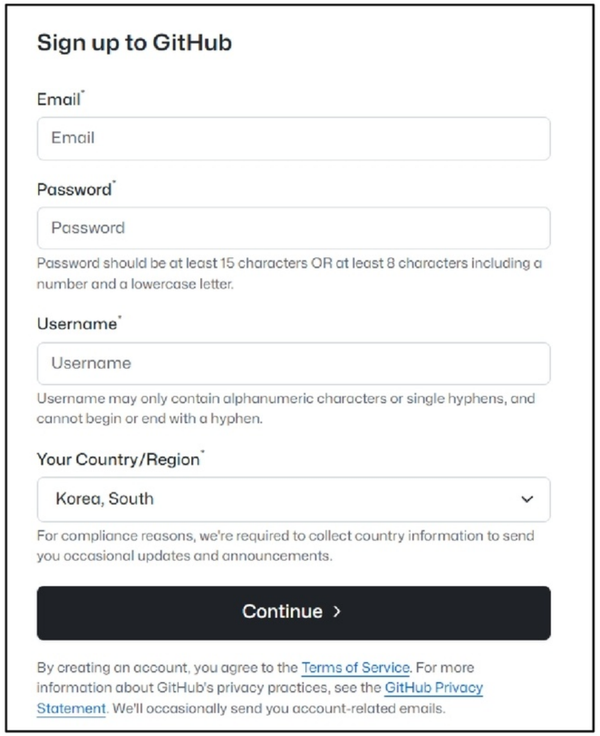
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

회원 가입

회원 가입

- User 객체를 Create 하는 과정
- 사용자로부터 아이디, 비밀번호 등의 정보를 입력 받아, DB에 새로운 User 객체를 생성하고 저장



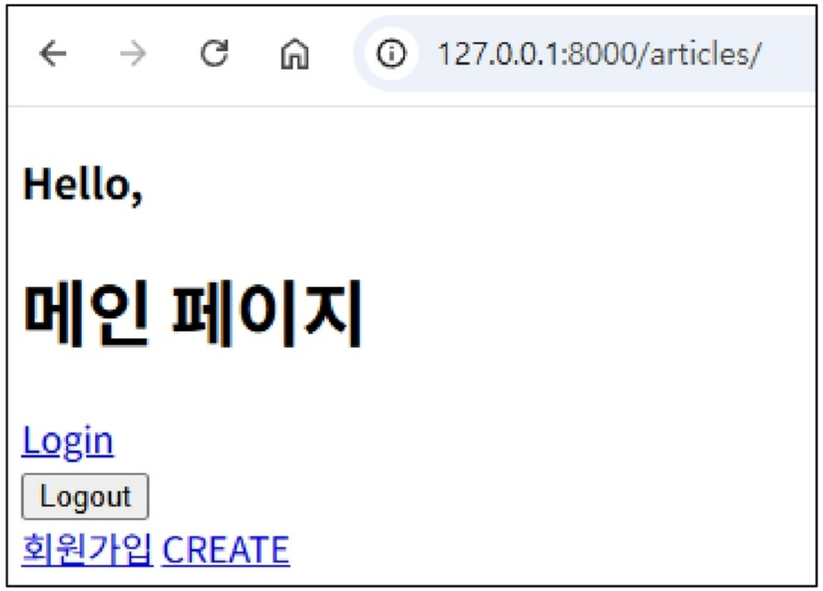
<그림9_Django_Authentication System 02_Github 회원가입 예시>

회원 가입 기능 구현 (1/3)

- 회원가입 경로 url 및 링크 생성

```python
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    ....
    path('signup/', views.signup, name='signup'),
]
```

```html
<!-- articles/index.html -->

<h1>메인 페이지</h1>
<a href="{% url "accounts:login" %}">Login</a>
<form action="{% url "accounts:logout" %}" method="POST">
  {% csrf_token %}
  <input type="submit" value="Logout">
</form>
<a href="{% url "accounts:signup" %}">회원가입</a>
```



<그림10_Django_Authentication System 02_회원가입 링크>

회원 가입 기능 구현 (2/3)

- .../accounts/signup/ url로 요청이 들어왔을 때 실행할 signup 함수 작성
- 회원가입에 사용할 데이터를 입력 받는 UserCreationForm built-in form 사용

```python
# accounts/views.py

from django.contrib.auth.forms import UserCreationForm


def signup(request):
    if request.method == 'POST':
        pass
    else:
        form = UserCreationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/signup.html', context)
```

UserCreationForm()

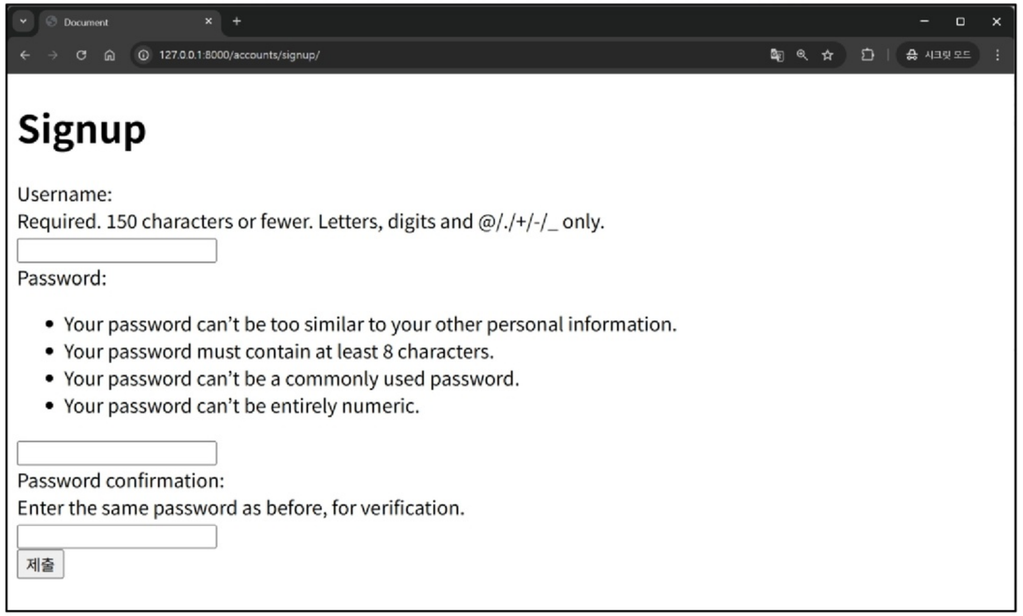
- 회원 가입시 사용자 입력 데이터를 받는 built-in ModelForm
- ModelForm이기 때문에, 유효성 검사를 통과한 데이터로 새로운 User 객체를 생성하고 데이터베이스에 저장하는 역할을 수행
- 이때 비밀번호는 자동으로 해시(단방향 변환)되어 저장됨 (자세한 원리는 다음 시간)

```python
class UserCreationForm(BaseUserCreationForm):
    def clean_username(self):
        """Reject usernames that differ only in case."""
        username = self.cleaned_data.get("username")
        if (
            username
            and self._meta.model.objects.filter(username__iexact=username).exists()
        ):
            self._update_errors(
                ValidationError(
                    {
                        "username": self.instance.unique_error_message(
                            self._meta.model, ["username"]
                        )
                    }
                )
            )
        else:
            return username
```

<그림11_Django_Authentication System 02_UserCreationForm>

회원 가입 기능 구현 (3/3)

- User 객체를 생성하는 요청이므로 POST 방식을 사용
- CSRF 공격을 방지하기 위해 csrf_token 작성
- 서버로부터 전달받은 UserCreationForm을 화면에 출력



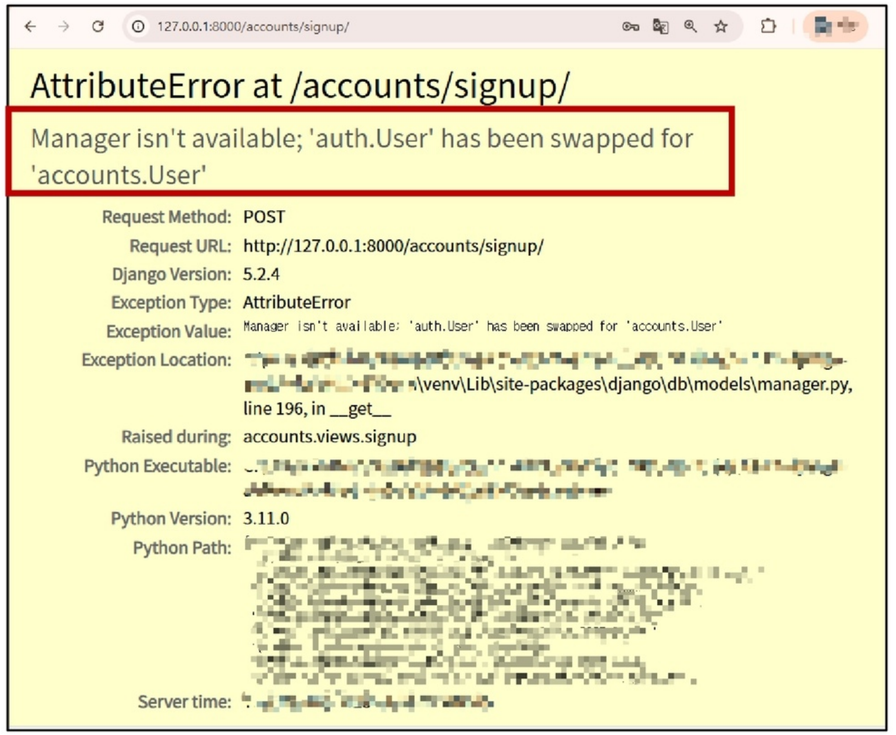
<그림10_Django_Authentication System 02_회원가입 페이지>

```
<!-- accounts/signup.html -->

<h1>Signup</h1>
<form action="{% url 'accounts:signup' %}" method="POST">
  {% csrf_token %}
  {{ form }}
  <input type="submit">
</form>
```

회원 가입 기능 에러 발생 및 해결 (1/2)

- 회원가입 시도 후 에러 페이지 확인

➢ `Manager isn't available; 'auth.User' has been swapped for 'accounts.User'`



<그림12_Django_Authentication System 02_회원가입 에러 화면>

TIP

- AI가 대부분의 디버깅을 도와주지만, 그럼에도 에러 페이지를 직접 확인하는 능력을 길러야 해요.
- 에러 메시지는 그 자체로 학습 자료입니다. 내가 놓친 개념을 다시 확인할 수 있기 때문이죠.

회원 가입 기능 에러 발생 및 해결 (2/2)

- 회원가입에 사용하는 UserCreationForm의 Meta.model이 기본 User(auth.User)로 고정되어 있기 때문

➢ model에 Custom User Model을 연결하자!

```python
class BaseUserCreationForm(SetPasswordMixin, forms.ModelForm):
    """
    A form that creates a user, with no privileges, from the given username and
    password.

    This is the documented base class for customizing the user creation form.
    It should be kept mostly unchanged to ensure consistency and compatibility.
    """

    password1, password2 = SetPasswordMixin.create_password_fields()

    class Meta:
        model = User
        fields = ("username",)
        field_classes = {"username": UsernameField}
```

https://github.com/django/django/blob/main/django/contrib/auth/forms.py#L210

<그림13_Django_Authentication System 02_BaseUserCreationForm>

TIP

AuthenticationForm은 왜 그대로 써도 됐을까?

- AuthenticationForm은 Form이라 특정 모델에 묶여 있지 않고, 내부에서 `get_user_model()`로 현재 User 모델을 찾아 씁니다.
- UserCreationForm은 ModelForm이라 Meta에 모델이 고정되어 있습니다. 그래서 Meta만 다시 써주면 됩니다.

커스텀 유저 모델을 사용을 하기 위해서 Form을 다시 작성

- Custom User model을 사용할 수 있도록 상속 후 일부분만 재작성

```python
# accounts/forms.py

from django.contrib.auth import get_user_model
from django.contrib.auth.forms import UserCreationForm


class CustomUserCreationForm(UserCreationForm):
    class Meta(UserCreationForm.Meta):
        model = get_user_model()
```

https://docs.djangoproject.com/en/5.2/topics/auth/customizing/#custom-users-and-the-built-in-auth-forms

get_user_model()

get_user_model()

"현재 프로젝트에서 활성화된 사용자 모델(active user model)"을 반환하는 함수

- 프로젝트 설정(AUTH_USER_MODEL)에 따라 기본 User 모델일 수도, 우리가 직접 만든 커스텀 User 모델일 수도 있기 때문에 올바른 모델을 동적으로 가져오기 위해 사용합니다.
- 모델을 직접 가져오는 대신 get_user_model()을 쓰면, User 모델이 바뀌어도 코드를 수정할 필요가 없어 재사용성과 유연성이 높아집니다.

User 모델을 직접 참조하지 않는 이유

- get_user_model()을 사용해 User 모델을 참조하면 커스텀 User 모델을 자동으로 반환해주기 때문
- Django는 필수적으로 User 클래스를 직접 참조하는 대신 get_user_model()을 사용해 참조해야 한다고 강조하고 있음

> **Referencing the User model** ¶
>
> If you reference **User** directly (for example, by referring to it in a foreign key), your code will not work in projects where the **AUTH_USER_MODEL** setting has been changed to a different user model.

<그림14_Django_Authentication System 02_Django 공식 User 모델 참조 안내>

※ User model 참조에 대한 자세한 내용은 추후 모델 관계에서 다룰 예정

회원 가입 로직 완성

- 내장 Form이었던 UserCreationForm 을 CustomUserCreationForm 으로 변경

```python
# accounts/views.py

from .forms import CustomUserCreationForm

def signup(request):
    if request.method == 'POST':
        form = CustomUserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('articles:index')
    else:
        form = CustomUserCreationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/signup.html', context)
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

회원 탈퇴

회원 탈퇴

- User 객체를 Delete 하는 과정
- `request.user.delete()` 를 활용해서 유저 객체 삭제를 진행

회원 탈퇴 로직 작성 (1/2)

- 회원 탈퇴 경로 url 및 링크 생성

```python
# accounts/urls.py

from django.urls import path
from . import views

app_name = 'accounts'
urlpatterns = [
    ...,
    path('delete/', views.delete, name='delete'),
]
```

```html
<!-- articles/index.html -->

<form action="{% url 'accounts:delete' %}" method="POST">
  {% csrf_token %}
  <input type="submit" value="회원탈퇴">
</form>
```

<그림15_Django_Authentication System 02_메인 페이지 회원탈퇴 링크>

회원 탈퇴 로직 작성 (2/2)

- .../accounts/delete/ url로 요청이 들어왔을 때 실행할 delete 메서드 작성
- 현재 로그인한 사용자 정보를 활용해 삭제하고, 메인 페이지로 이동

```python
# accounts/views.py

def delete(request):
    request.user.delete()
    return redirect('articles:index')
```

인증된 사용자에 대한 접근 제한

인증된 사용자에 대한 접근 제한

1. `is_authenticated` 속성

2. `login_required` 데코레이터

is_authenticated 속성

1. is_authenticated 속성

- 사용자가 인증 되었는지 여부를 알 수 있는 User model의 읽기 전용 속성
- 인증 사용자에 대해서는 항상 True, 비인증 사용자에 대해서는 항상 False
- 사용되는 경우
  - 사용자의 로그인 상태에 따라 다른 메뉴를 보여줄 때
  - view 함수 내에서 특정 기능을 로그인한 사용자에게만 허용하고 싶을 때

is_authenticated 적용하기 (1/2)

- 로그인과 비로그인 상태에서 화면에 출력되는 링크를 다르게 설정하기

```
<!-- articles/index.html -->

{% if request.user.is_authenticated %}
  <h3>Hello, {{ user.username }}</h3>
  <a href="{% url 'articles:create' %}">NEW</a>
  <form action="{% url 'accounts:logout' %}" method="POST">
    {% csrf_token %}
    <input type="submit" value="Logout">
  </form>
  <form action="{% url 'accounts:delete' %}" method="POST">
    {% csrf_token %}
    <input type="submit" value="회원탈퇴">
  </form>
{% else %}
  <a href="{% url 'accounts:login' %}">Login</a>
  <a href="{% url 'accounts:signup' %}">Signup</a>
{% endif %}
```

is_authenticated 적용하기 (2/2)

- 인증된 사용자라면 로그인/회원가입 로직을 수행할 수 없도록 하기

```
# accounts/views.py

def login(request):
    if request.user.is_authenticated:
        return redirect('articles:index')
    …

def signup(request):
    if request.user.is_authenticated:
        return redirect('articles:index’)
    …
```

login_required 데코레이터

2. `login_required` 데코레이터

- 인증된 사용자에 대해서만 view 함수를 실행시키는 데코레이터
- 비인증 사용자의 경우 `/accounts/login/` 주소로 redirect 시킴
- 사용되는 경우
  - 게시글 작성, 댓글 달기 등 누가 작성했는지 중요한 곳에서 사용

데코레이터
기존 함수를 감싸, 특별한
기능(인증 등)을 추가하는 함수.

<개념3_Django_Authentication System 02_데코레이터>

login_required 적용하기 (1/2)

- 인증된 사용자만 게시글을 작성/수정/삭제 할 수 있도록 수정

```python
# articles/views.py

from django.contrib.auth.decorators import login_required


@login_required
def create(request):
    pass

@login_required
def delete(request, article_pk):
    pass

@login_required
def update(request, article_pk):
    pass
```

login_required 적용하기 (2/2)

- 인증된 사용자만 로그아웃/탈퇴/수정/비밀번호 변경 할 수 있도록 수정

```python
# accounts/views.py

from django.contrib.auth.decorators import login_required

@login_required
def logout(request):
    pass

@login_required
def delete(request):
    pass
```

```python
# accounts/views.py

…

@login_required
def update(request):
    pass

@login_required
def password(request):
    pass
```

참고

is_authenticated 코드

is_authenticated 코드

- 메서드가 아닌 속성 값임을 주의

```
@property
def is_authenticated(self):
    """
    Always return True. This is a way to tell if the user has been
    authenticated in templates.
    """
    return True
```

<그림16_Django_Authentication System 02_is_authenticated>

https://github.com/django/django/blob/main/django/contrib/auth/base_user.py#L85

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

회원가입 후 자동 로그인

회원가입 후 로그인까지 이어서 진행하려면?

- 회원가입 성공한 user 객체를 활용해 login 진행

```python
# accounts/views.py

def signup(request):
    if request.method == 'POST':
        form = CustomUserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            auth_login(request, user)
            return redirect('articles:index')
    else:
        form = CustomUserCreationForm()
    context = {
        'form': form,
    }
    return render(request, 'accounts/signup.html', context)
```

```python
def save(self, commit=True):
    user = super().save(commit=False)
    user.set_password(self.cleaned_data["password1"])
    if commit:
        user.save()
    return user
```

<그림17_Django_Authentication System 02_save 메서드>

※ UserCreationForm의 save 메서드

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

회원 탈퇴 개선

탈퇴와 함께 기존 사용자의 Session Data 삭제 방법

- 사용자 객체 삭제 이후 로그아웃 함수를 호출해야 함
- 단, "탈퇴(1) 후 로그아웃(2)"의 순서가 바뀌면 안됨
  - 먼저 로그아웃하면 `request.user`가 AnonymousUser(비로그인 사용자)로 바뀌기 때문에, 그 뒤의 `request.user.delete()`는 지울 대상이 없어 에러가 남

```python
# accounts/views.py

@login_required
def delete(request):
    request.user.delete()

    auth_logout(request)

    return redirect('articles:index')
```

Auth built-in form 코드

Auth built-in form github 코드

- UserCreationForm()
  - https://github.com/django/django/blob/5.2/django/contrib/auth/forms.py#L271

```
class UserCreationForm(BaseUserCreationForm):
    def clean_username(self):
        """Reject usernames that differ only in case."""
        username = self.cleaned_data.get("username")
        if (
            username
            and self._meta.model.objects.filter(username__iexact=username).exists()
        ):
            self._update_errors(
                ValidationError(
                    {
                        "username": self.instance.unique_error_message(
                            self._meta.model, ["username"]
                        )
                    }
                )
            )
        else:
            return username
```

<그림11_Django_Authentication System 02_UserCreationForm>

실습

로그아웃

- 3042. 로그아웃 기능 구현과 코드 개선
- 3242. 로그인 로그아웃 구현하기
- 3244. 랭킹 관리 시스템 만들기 - 로그인과 로그아웃

회원가입

- 3043. 회원가입 기능 구현 - 준비
- 3248. 회원 관리 시스템 만들기 - 로그인, 회원가입

인증된 사용자에 대한 접근 제한

- 3251. 회원 정보 관리 서비스 만들기 - 로그인 여부 판단하기

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 로그아웃의 역할로 가장 올바른 것은?

a) 세션 데이터만 삭제
b) 클라이언트 쿠키만 삭제
c) 세션 데이터와 쿠키 모두 처리
d) 사용자 계정 정보 삭제

2 회원가입 기능에 사용하는 Django 내장 폼은?

a) `AuthenticationForm`
b) `UserCreationForm`
c) `UserChangeForm`
d) `CustomUserForm`

3 get_user_model() 함수를 사용하는 가장 큰 이유는?

a) 사용자 객체를 삭제하기 위해
b) 비밀번호를 암호화하기 위해
c) 활성화된 User 모델을 참조하기 위해
d) 모든 사용자 정보를 조회하기 위해

4 로그인된 사용자의 회원 탈퇴를 구현하는 코드는?

a) `request.user.delete()`
b) `User.objects.delete(request.user)`
c) `delete(request.user)`
d) `request.user.remove()`

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 템플릿에서 사용자의 로그인 여부를 확인하는 속성은?

a) is_active
b) is_staff
c) is_loggedin
d) is_authenticated

6 @login_required 데코레이터의 역할은 무엇인가요?

a) 로그인 페이지를 렌더링
b) 비인증 사용자의 접근을 제한
c) 모든 사용자의 접근을 허용
d) 관리자만 접근을 허용

7 회원가입 직후 자동 로그인을 시키는 방법은?

a) form.save() 후 login() 호출
b) login() 호출 후 form.save()
c) form.save()만 호출
d) login()만 호출

8 회원 탈퇴 시 올바른 처리 순서는?

a) 로그아웃 후 계정 삭제
b) 계정 삭제 후 로그아웃
c) 순서는 상관 없음
d) 동시에 처리해야 함

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 AbstractUser와 AbstractBaseUser의 주요 차이점은?

a) AbstractUser가 더 자유도가 높다

b) AbstractBaseUser는 필드가 더 많다

c) AbstractUser는 기본 필드를 포함한다

d) 둘은 기능적으로 완전히 동일하다

10 로그아웃 요청은 어떤 방식으로 보내야 안전한가요?

a) GET

b) POST

c) PUT

d) DELETE

확인 문제

정답 및 해설

1. c) 세션 데이터와 쿠키 모두 처리 2. b) `UserCreationForm` 3. c) 활성화된 User 모델을 참조하기 위해 4. a) `request.user.delete()`
5. d) `is_authenticated` 6. b) 비인증 사용자의 접근을 제한 7. a) `form.save()` 후 `login()` 호출 8. b) 계정 삭제 후 로그아웃

1. 로그아웃은 서버의 세션 데이터를 삭제하고, 클라이언트의 쿠키에 저장된 세션 ID도 삭제하여 연결을 완전히 끊는 과정입니다.
2. UserCreationForm은 새로운 사용자를 생성(회원가입)하기 위해 Django에서 기본으로 제공하는 ModelForm입니다.
3. 프로젝트의 User 모델이 기본이든 커스텀이든, 현재 활성화된 모델을 동적으로 가져와 코드의 유연성과 재사용성을 높입니다.
4. request 객체에 담겨있는 현재 로그인된 사용자(user) 객체의 delete() 메소드를 호출하여 간단하게 탈퇴시킬 수 있습니다.
5. is_authenticated 속성은 사용자가 인증(로그인)되었는지 여부를 True/False로 반환하는 가장 확실한 방법입니다.
6. 인증되지 않은 사용자가 해당 view에 접근하면, 지정된 로그인 페이지로 리디렉션시켜 접근을 막습니다.
7. form.save()가 반환하는 user 객체를 login() 함수에 인자로 넘겨주면 세션이 생성되어 자동 로그인됩니다.
8. 계정을 먼저 삭제한 뒤, 남아있는 세션 정보를 로그아웃으로 정리해야 합니다. 반대로 하면 어떤 계정을 지워야 할지 알 수 없게 됩니다.

확인 문제

정답 및 해설

9. c) `AbstractUser`는 기본 필드를 포함한다 10. b) POST

9. AbstractUser는 기본 User 모델의 모든 필드를 포함하여 확장이 용이하고, AbstractBaseUser는 최소한의 필드만 제공하여 자유도가 높습니다.
10. GET 요청은 데이터 변경을 유발해서는 안 되므로, 서버 상태를 변경하는 로그아웃은 POST 방식으로 처리해야 안전합니다.

활동 정리

핵심 키워드

개념	설명	예시
로그아웃	DB 세션과 클라이언트 쿠키를 삭제	`auth_logout(request)`
회원가입 폼	사용자 생성을 위한 내장 ModelForm	`form = UserCreationForm()`
활성 유저 모델	현재 프로젝트의 활성 유저 모델 반환	`model = get_user_model()`
인증 확인 속성	사용자 인증 여부를 확인하는 속성	`{% if user.is_authenticated %}`
로그인 요구	인증된 사용자만 view를 실행	`@login_required`
회원 탈퇴	현재 로그인된 User 객체를 삭제	`request.user.delete()`

<표2_Django_Authentication System 02_핵심 키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- 로그아웃
  - 로그아웃은 세션(Session)을 삭제하는 과정
  - 서버의 세션 데이터를 지우고, 클라이언트 브라우저에 있는 세션 쿠키를 삭제
  - logout 함수를 사용하며, 이 함수는 현재 요청에 대한 세션 데이터를 DB에서 삭제하고 클라이언트의 쿠키에서도 세션 ID를 제거
  - 로그아웃 요청은 보통 POST 방식으로 처리하여 CSRF 공격을 방지함

활동 정리

오늘 공부한 내용 요약 및 정리

- 회원 가입
  - 회원 가입은 사용자(User) 객체를 생성(Create)하는 과정
  - 사용자의 정보를 입력받아 데이터베이스에 새로운 User 객체를 만들고 저장
  - UserCreationForm은 회원 가입 시 사용자 입력을 받는 ModelForm
  - 이 폼은 유효성 검사를 통과한 데이터로 새로운 User 객체를 생성하고, 비밀번호를 자동으로 암호화하여 저장함
  - Custom User Model을 활용한 회원가입
    - 커스텀 유저 모델을 사용할 때는 UserCreationForm을 그대로 사용하면 에러가 발생함
    - 따라서 UserCreationForm을 상속받는 CustomUserCreationForm을 직접 만들어야 함

활동 정리

오늘 공부한 내용 요약 및 정리

- 회원 탈퇴
  - 회원 탈퇴는 User 객체를 삭제(Delete)하는 과정
  - `request.user.delete()` 코드를 통해 로그인한 사용자의 객체를 삭제
  - 실제 서비스에서는 데이터를 바로 삭제하기보다 계정을 비활성화 처리하는 경우가 많음
  - 사용자를 삭제한 후에는 로그아웃 함수를 호출하여 세션 데이터를 정리해야 함
  - "탈퇴 후 로그아웃" 순서를 지켜야 함. 반대로 하면 요청 객체 정보가 사라져 탈퇴할 유저 정보를 찾을 수 없게 됨

활동 정리

오늘 공부한 내용 요약 및 정리

- 인증된 사용자의 접근 제한
  - `is_authenticated` 속성
    - is_authenticated는 사용자의 인증 여부를 알려주는 User 모델의 읽기 전용 속성
    - 인증된 사용자는 항상 True, 비인증 사용자는 항상 False 값을 가짐
    - 템플릿에서 사용자의 로그인 상태에 따라 다른 메뉴를 보여줄 때 유용
  - `login_required` 데코레이터
    - login_required는 인증된 사용자에 대해서만 view 함수를 실행시키는 데코레이터
    - 비인증 사용자가 이 데코레이터가 적용된 view에 접근하면, 설정된 로그인 페이지로 리디렉션됨

활동 정리

프리미엄 회원을 위한 도서 웹사이트를 만들려고 하는데, 로그인 기능만으로는 부족합니다. 무엇이 더 필요할까요?

1. 비회원
   - 회원가입이나 로그인 페이지를 보여줘야 합니다.
2. 회원
   - 언제든 로그아웃할 수 있어야 하며, 회원 탈퇴를 통해 스스로 삭제할 권리를 보장해야 합니다.
   - 로그인 여부에 따라 다른 화면 구성해야 합니다.

- 회원가입(UserCreationForm) 과 로그인(AuthenticationForm)을 활용해서 손쉽게 페이지를 구현할 수 있다.
- logout(request) 와 request.user.delete() 를 활용하여 로그아웃/회원 탈퇴를 구현할 수 있다.
- is_authenticated 속성과 login_required 데코레이터를 활용하여 권한별 다른 화면 구성을 할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다